

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №3
За седьмой семестр
По дисциплине: «Модели решения задач в интеллектуальных системах»
Тема: Сравнение Сигмоиды и ReLU в многослойном персептроне для
задачи XOR

Выполнил:
Студент 4 курса
Группы ИИ-26(2)
Семёнов А.В.
Проверил:
Никитин С.К.

Брест 2026

Сравнение Сигмоиды и ReLU в задаче XOR

1 Условие работы и вариант задания

Цель работы — исследовать влияние функции ReLU в скрытом слое многослойного персептрона 2–2–1 на скорость и устойчивость обучения, точность и форму разделяющей поверхности. За основу взята сеть с BCE из лабораторной работы №2. Вычисления прямого прохода, градиентов и обновлений реализованы вручную на Python с NumPy; графики построены с Matplotlib.

Вариант 1: $c_0 = 0$, $c_1 = -6$. Обучение проводится на полной таблице истинности XOR. Сравниваются конфигурация А — Сигмоида в скрытом и выходном слоях — и конфигурация В — ReLU в скрытом слое и Сигмоида на выходе. Для обеих сетей используется бинарная кросс-энтропия BCE.

А	В	Цель y	Метка t
0	0	0	0
0	-6	-6	1
-6	0	-6	1
-6	-6	0	0

Нормализация и восстановление шкалы

$$x_1 = -A/6; \quad x_2 = -B/6; \quad t = -y/6$$
$$p = \sigma(z); \quad \hat{y} = c_0 + (c_1 - c_0)p = -6p$$

Входы и цели переводятся в шкалу 0/1, поскольку сигмоидный выход не может непосредственно принимать значение -6. Восстановленный выход лежит между -6 и 0. Класс определяется по ближайшему из чисел 0 и -6; при равенстве расстояний выбирается 0. Порог класса: $p = 0,5$, или $\hat{y} = -3$.

Для произвольного ввода из $[-10; 10]$ нормализация выполняется без округления и без обрезания. За пределами $[-6; 0]$ по любой координате сеть экстраполирует. Эталон XOR задан только для четырёх обучающих пар, поэтому дополнительные примеры не включаются в ассигасу.

2 Архитектура и алгоритм обучения

Сеть содержит два входа, два скрытых нейрона и один выходной нейрон: четыре веса W_{ij} , два скрытых смещения b_j , два выходных веса v_j и выходное смещение b_o . Всего обучаются девять параметров.

$$\begin{aligned}a_j &= \sum_i x_i W_{ij} + b_j; \quad h_j = f(a_j); \quad z = \sum_j h_j v_j + b_o \\ \sigma(a) &= 1/(1 + \exp(-a)); \quad \sigma'(a) = \sigma(a)(1 - \sigma(a)) \\ \text{ReLU}(a) &= \max(0, a); \quad \text{ReLU}'(a) = 1 \text{ при } a > 0, \text{ иначе } 0\end{aligned}$$

В точке $a = 0$ у ReLU нет обычной производной; в программе выбран нулевой субградиент. Выходная активация в обеих конфигурациях — Сигмоида.

Обратное распространение ошибки

$$\begin{aligned}L &= -t \ln(p) - (1 - t) \ln(1 - p); \quad Es = \sum_k L_k \\ L &= \max(z, 0) - tz + \ln(1 + \exp(-|z|)) \\ \delta_o &= p - t; \quad \delta_j = \delta_o v_j f'(a_j) \\ \partial L / \partial W_{ij} &= x_i \delta_j; \quad \partial L / \partial b_j = \delta_j \\ \partial L / \partial v_j &= h_j \delta_o; \quad \partial L / \partial b_o = \delta_o; \quad \theta \leftarrow \theta - \eta \partial L / \partial \theta\end{aligned}$$

ВСЕ вычисляется устойчиво через логит z . Все градиенты получаются со старыми весами, затем параметры обновляются после каждого примера. По завершении эпохи Es заново рассчитывается на всех четырёх точках при единых текущих весах. Остановка: $Es \leq Ee$ либо предел эпох.

Параметр	Значение
Скорость обучения η	0,5
Порог Ee	0,01
Максимум эпох	50 000
Seed	42, 43, 44, 45, 46
Инициализация	default_rng; U[−0,5; 0,5)
Порядок примеров	Фиксированный, как в таблице XOR

Параметры и порядок генерации весов сохранены из ЛР №2. При каждом seed начальные веса A и B совпадают. Перезапуски отсутствуют; для карт и режима функционирования заранее выбран первый seed = 42, включая его неудачный результат ReLU.

3 Результаты серии экспериментов

Результаты получены выполнением lab3.py с параметрами по умолчанию, NumPy 2.3.5. Ассигасу — доля верных ближайших классов на четырёх примерах; $MAE = \frac{1}{4}\sum|\hat{y} - y|$. Все показатели рассчитаны по конечным весам.

Сеть	Seed	Эпохи	Es	Accuracy	MAE	Es ≤ Ee
A	42	2331	0.009996476	100%	0.014975	Да
A	43	2418	0.009998959	100%	0.014979	Да
A	44	2279	0.009996110	100%	0.014975	Да
A	45	2439	0.009997042	100%	0.014976	Да
A	46	50000	1.390927398	75%	1.500221	Нет
B	42	50000	1.920094090	75%	2.060374	Нет
B	43	50000	1.909582328	75%	1.996500	Нет
B	44	50000	1.909580328	75%	1.996496	Нет
B	45	50000	1.909583096	75%	1.996502	Нет
B	46	50000	1.909580596	75%	1.996497	Нет

Показатель	A Сигмоида	B ReLU
Достигли порога	4 из 5	0 из 5
Эпохи успешных, среднее ± σ	2366,75 ± 64,86	Нет успешных
Диапазон эпох успешных	2279–2439	Не определён
Выполнено эпох, среднее	11 893,4	50 000
Средняя Es	0.286183	1.911684
Средняя ассигасу	95%	75%
MAE, среднее ± σ	0,312025 ± 0,594098	2,009274 ± 0,025550

σ — стандартное отклонение по совокупности пяти опытов (ddof = 0); для числа эпох успешных используются только четыре успешных опыта A. Средние Es, ассигасу и MAE учитывают все пять запусков каждой конфигурации. 50 000 — число выполненных эпох неудачного запуска, а не время достижения порога.

В этой серии Сигмоида оказалась надёжнее: четыре достижения порога против нуля. Для ReLU время сходимости при данных настройках не измерено, поэтому сравнение средних времён только успешных запусков между A и B невозможно. Итоговые 75% ReLU означают одну ошибку в таблице из четырёх строк.

4 Скорость обучения и устойчивость

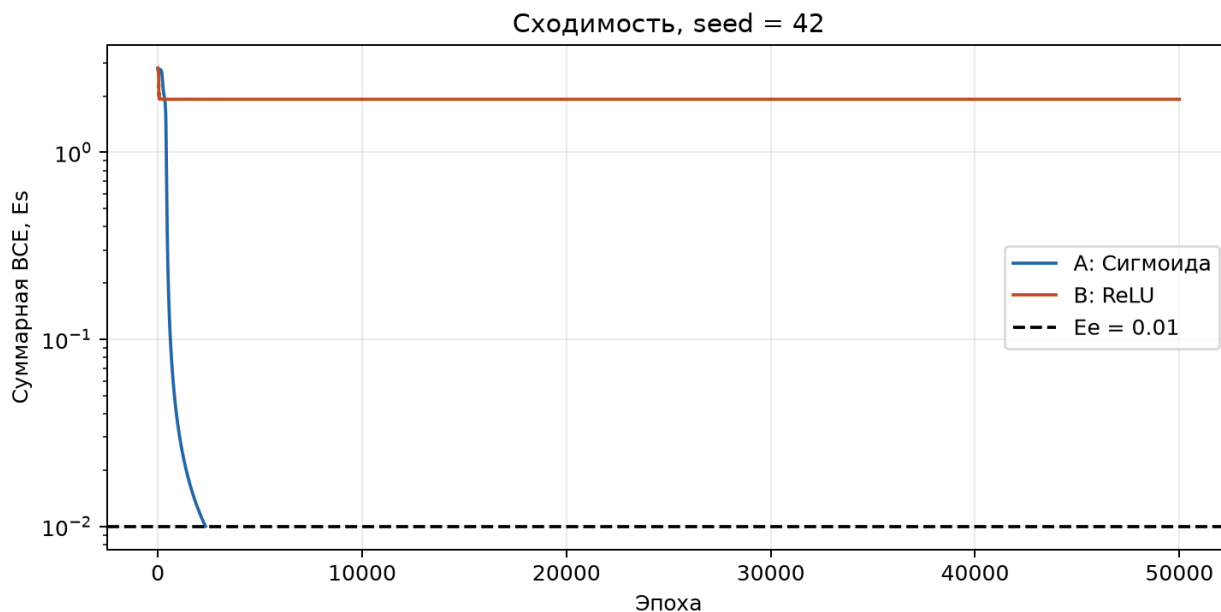


Рисунок 1 — Суммарная BCE при seed 42

А достигла Ee на эпохе 2331. В остановилась по лимиту с $E_s = 1,920094090$. Для обеих кривых используются одна функция ошибки и один порог; вертикальная шкала логарифмическая.

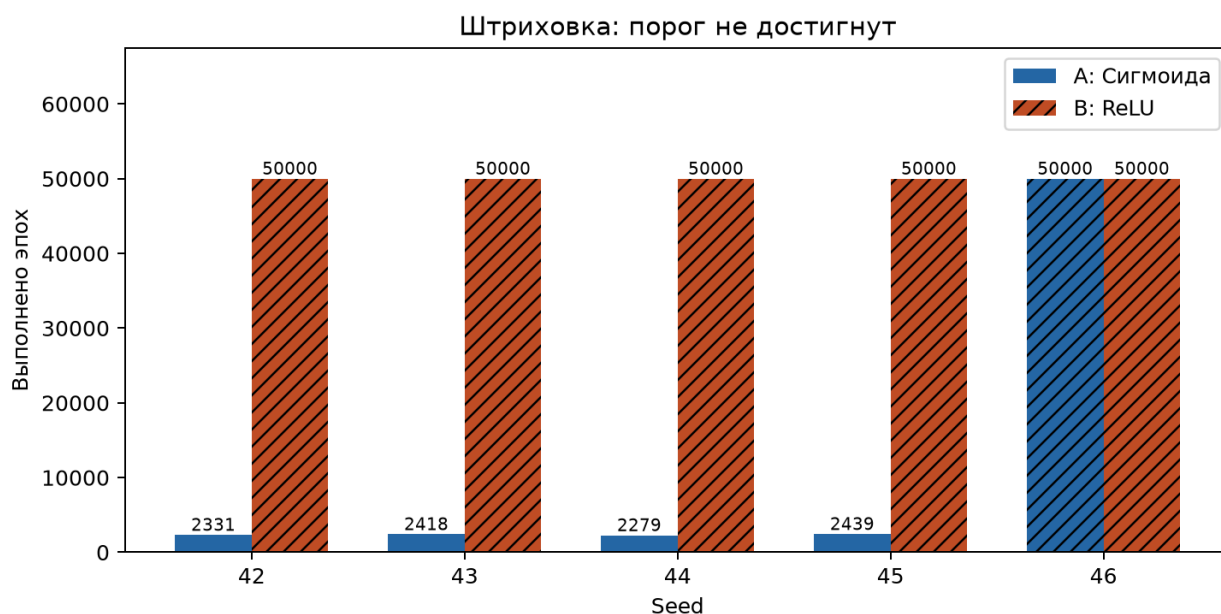


Рисунок 2 — Число выполненных эпох в пяти запусках

Штриховкой показаны запуски, не достигшие критерия. Сигмоида при seed = 46 сохранила accuracy = 75%; ReLU — 75% при каждом seed. Пять опытов характеризуют данную серию, а не общую вероятность успешного обучения.

5 Разделяющая поверхность и качество выхода

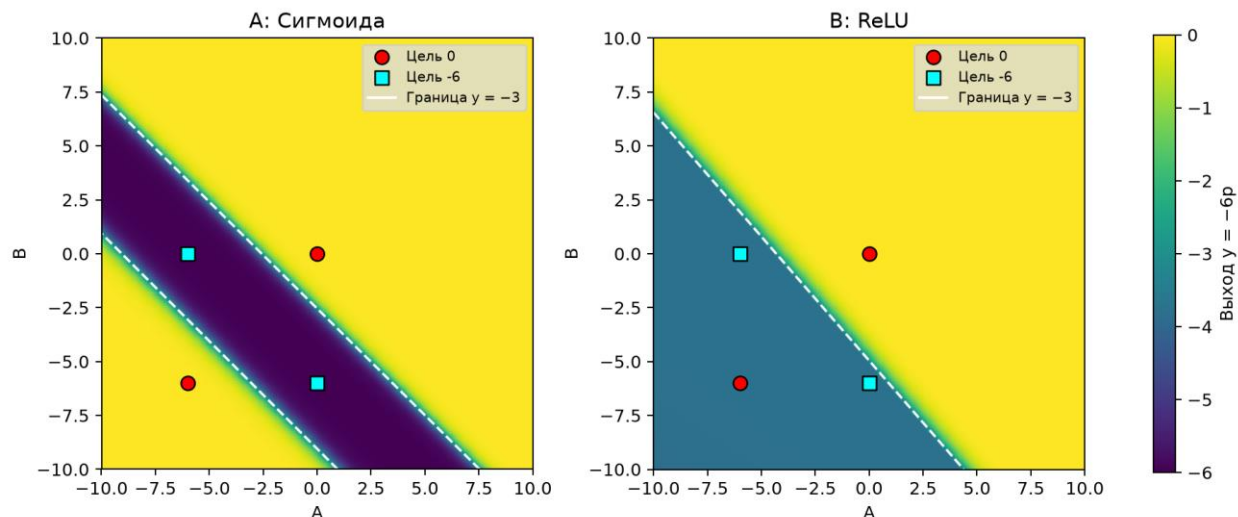


Рисунок 3 — Карты выхода A и B при seed 42

Общая цветовая шкала показывает $\hat{y}$ от -6 до 0 . Точки обозначают истинные классы, белая линия — $\hat{y} = -3$. Сигмоидная сеть выделяет диагональную полосу и правильно классифицирует все четыре вершины. У ReLU при seed = 42 один скрытый нейрон неактивен на выборке: итоговая граница не отделяет $(-6; -6)$ от двух примеров класса -6 .

Для ReLU логит z кусочно-аффинен по входам, поэтому граница $z = 0$ состоит из прямолинейных фрагментов. Выход $p = \sigma(z)$ сам по себе не является кусочно-линейным. На неудачной карте видна фактически одна прямая граница. У Сигмоиды поверхность гладкая; её границы на данном масштабе могут выглядеть почти прямыми. XOR не задаёт единственную правильную границу вне четырёх обучающих точек.

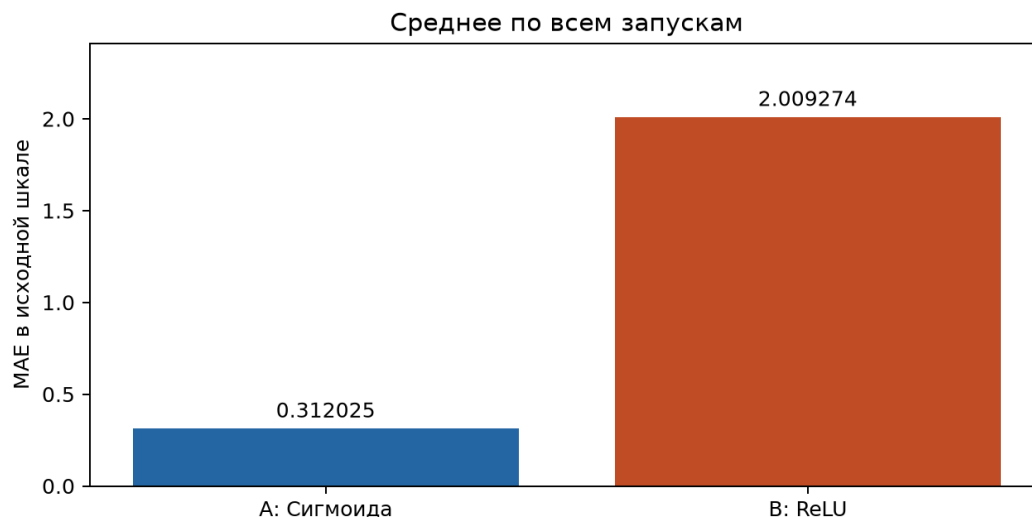


Рисунок 4 — Средняя MAE по всем пяти запускам

Одинаковая BCE и одинаковый E_e позволяют сопоставлять точность остановки. Из $-\ln(1 - e) \geq e$ при $e = |p - t|$ следует $MAE \leq 6E_e/4$. Поэтому успешная остановка при $E_e = 0,01$ гарантирует $MAE \leq 0,015$; неудачные запуски этой гарантии не имеют.

6 Режим функционирования и диагностика

При $\text{seed} = 42$ программа демонстрирует четыре обучающие и три дополнительные пары. Для каждой конфигурации выводятся p , $\hat{y}$ и ближайший числовой класс.

A; B	p A	$\hat{y}$ A	Класс A	p B	$\hat{y}$ B	Класс B
0; 0	0.002118	-0.012707	0	0.000003	-0.000016	0
0; -6	0.997766	-5.986596	-6	0.626420	-3.758522	-6
-6; 0	0.997745	-5.986471	-6	0.626420	-3.758522	-6
-6; -6	0.003377	-0.020261	0	0.626420	-3.758522	-6
-1; -5	0.997763	-5.986576	-6	0.626420	-3.758522	-6
-1; -1	0.094950	-0.569702	0	0.000705	-0.004233	0
-5; -5	0.055700	-0.334198	0	0.626420	-3.758522	-6

Неактивные нейроны ReLU

Seed	До обучения	Первая эпоха	После обучения
42	Нет	1	1
43	Нет	1	2
44	1	0	1
45	Нет	Нет	Нет
46	2	0	2

Нейрон считается неактивным на обучающей выборке, если $a_j \leq 0$ на всех четырёх точках; нумерация начинается с 1. Такие нейроны получают нулевой скрытый градиент и при данном алгоритме не оживают. При $\text{seed} = 44$ и 46 это произошло уже при инициализации, при 42 и 43 — после первой эпохи. При $\text{seed} = 45$ полностью неактивных нейронов нет, но обучение также не удалось: гибель нейронов не является единственной причиной неудачи.

Проверка корректности

Все девять градиентов проверены центральными конечными разностями с шагом 10^{-6} для обеих активаций и обеих меток. Для ReLU проверены активная и неактивная ветви вдали от излома. Максимальное расхождение $7,260 \cdot 10^{-11}$ при допуске 10^{-7} . Отдельно проверены выбранная производная в нуле, воспроизводимость, одинаковая инициализация, ВСЕ при логитах ± 1000 , остановка на нулевой эпохе, границы ввода и отклонение NaN, бесконечности и выхода за диапазон.

7 Выводы

Скорость и устойчивость. При $\eta = 0,5$ Сигмоида достигла порога в 4/5 запусков, в среднем за 2366,75 эпох среди успешных; ReLU — в 0/5. В данном эксперименте преимущество по достижению критерия у Сигмоиды. Универсального превосходства это не доказывает: результат зависит от шага, начальных весов и порядка примеров.

Форма границы. У ReLU логит кусочно-аффинен, и его нулевой уровень имеет прямолинейные фрагменты. Сигмоидная сеть задаёт гладкое отображение. На карте ReLU при $\text{seed} = 42$ видна вырожденная граница с одной прямой; ожидаемая геометрия ReLU не означает правильного решения XOR.

Почему ReLU используют в глубоких сетях. На положительной ветви производная ReLU равна 1, тогда как производная Сигмоиды не превышает $1/4$ и стремится к нулю в насыщении. ReLU уменьшает этот источник затухания градиента, но на отрицательной ветви градиент равен нулю. На небольшой сети XOR с прежним шагом преимущество ReLU не подтвердилось.

Способность представить XOR. Обе архитектуры 2–2–1 способны представить XOR. Конструктивный пример для ReLU: $s = x_1 + x_2$, $h_1 = \max(0, s)$, $h_2 = \max(0, s - 1)$, $z = 20h_1 - 40h_2 - 10$. Для $s = 0, 1, 2$ получаются логиты $-10, 10, -10$ и верные классы. Этот пример проверяет выразительность и не подменяет результаты случайного обучения.

Мёртвые нейроны. В четырёх запусках В обнаружен один нейрон, неактивный на всей выборке. Его нулевой градиент блокирует обучение соответствующего скрытого признака. При $\text{seed} = 45$ оба нейрона активны хотя бы на одной точке, но этого недостаточно для успешной оптимизации.

Необходимость скрытого слоя. XOR линейно неразделим. Один сигмоидный нейрон имеет прямую границу и не отделяет противоположные вершины квадрата. Замена активации не устраняет потребности в нелинейном преобразовании признаков скрытым слоем.

Соответствие визуализации XOR. При $\text{seed} = 42$ карта А согласуется со всеми четырьмя метками, карта В ошибается на $(-6; -6)$. За пределами таблицы эталон не задан. Для повышения успешности ReLU можно исследовать меньший шаг и другую инициализацию; в представленной серии параметры намеренно сохранены одинаковыми.

8 Запуск программы

Программа полностью размещена в одном файле lab3.py и не зависит от файлов предыдущих лабораторных работ. Требуются Python 3.10 или новее, NumPy и Matplotlib.

```
python -m pip install numpy matplotlib
```

```
python lab3.py
```

```
python lab3.py --self-test
```

```
python lab3.py --no-input --no-show
```

Обычный запуск выполняет десять обучений, печатает результаты и показывает четыре графика. После закрытия окон доступен ввод пар A B через пробел. Допускаются десятичная точка и запятая; q завершает работу. Обработка некорректного ввода не прерывает программу.

Рядом со скриптом создаётся папка lab3_results: results.json содержит параметры, истории, веса, диагностику и прогнозы; summary.csv — сводную статистику; convergence.png, epochs.png, surfaces.png и mae.png — графики. Флаг --out задаёт другой каталог.

Параметры --seeds, --lr, --ee и --max-epochs позволяют повторить исследование с другими настройками. Нужно указать не менее пяти различных неотрицательных seed. При изменении параметров числа будут отличаться от отчёта. Представителем для карт и интерактивного режима всегда остаётся первый указанный seed.

Приложение А Полный код программы

"""ЛР 3, вариант 1: XOR, сигмоида и ReLU, онлайн-BCE.

Установка: python -m pip install numpy matplotlib

Запуск: python lab3.py

Проверки: python lab3.py --self-test

Без окон и ввода: python lab3.py --no-show --no-input

Все вычисления реализованы здесь; lab1.py и lab2.py не нужны.

"""

import argparse

import csv

import json

from pathlib import Path

import numpy as np

C0, C1 = 0., -6.

RAW_X = np.array([[0., 0.], [0., -6.], [-6., 0.], [-6., -6.]])

RAW_Y = np.array([0., -6., -6., 0.])

CHECK_X = np.vstack((RAW_X, [[-1., -5.], [-1., -1.], [-5., -5.])))

NAMES = {'sigmoid': 'A: Сигмоида', 'relu': 'B: ReLU'}

def normalize(v):

return (np.asarray(v, dtype=float) - C0) / (C1 - C0)

def decode(v):

return C0 + (C1 - C0) * np.asarray(v)

def sigmoid(v):

v = np.asarray(v, dtype=float)

e = np.exp(-np.abs(v))

return np.where(v >= 0, 1. / (1. + e), e / (1. + e))

def nearest(v):

v = np.asarray(v)

return np.where(np.abs(v - C0) <= np.abs(v - C1), C0, C1)

class MLP:

def __init__(self, activation, seed):

if activation not in NAMES:

raise ValueError('Неизвестная активация')

self.activation = activation

rng = np.random.default_rng(seed)

self.w1 = rng.uniform(-.5, .5, (2, 2))

self.b1 = rng.uniform(-.5, .5, 2)

self.w2 = rng.uniform(-.5, .5, 2)

self.b2 = np.array(rng.uniform(-.5, .5))

def forward(self, x):

a = np.asarray(x) @ self.w1 + self.b1

h = sigmoid(a) if self.activation == 'sigmoid' else np.maximum(0., a)

z = h @ self.w2 + self.b2

return a, h, z, sigmoid(z)

def parameters(self):

return self.w1, self.b1, self.w2, self.b2

def gradients(self, x, target):

a, h, _, p = self.forward(x)

delta = p - target # BCE + сигмоида на выходе

derivative = h * (1. - h) if self.activation == 'sigmoid' else (a > 0.)

В нуле выбираем производную ReLU, равную 0.

hidden_delta = delta * self.w2 * derivative

return np.outer(x, hidden_delta), hidden_delta, delta * h, np.asarray(delta)

def step(self, x, target, lr):

Все градиенты вычислены до изменения любого параметра.

```

        grads = self.gradients(x, target)
        for param, grad in zip(self.parameters(), grads):
            param -= lr * grad

def dead_units(self):
    # Номера нейронов, не активных ни на одной обучающей точке.
    if self.activation != 'relu':
        return []
    a = self.forward(normalize(RAW_X))[0]
    return (np.flatnonzero(np.all(a <= 0., axis=0)) + 1).tolist()

def loss_sum(model, x, targets):
    z = model.forward(x)[2]
    return float(np.sum(np.maximum(z, 0.) - targets * z
                             + np.log1p(np.exp(-np.abs(z)))))

def metrics(model):
    p = model.forward(normalize(RAW_X))[3]
    real = decode(p)
    return {'Es': loss_sum(model, normalize(RAW_X), normalize(RAW_Y)),
            'accuracy': float(np.mean(nearest(real) == RAW_Y)),
            'mae': float(np.mean(np.abs(real - RAW_Y))),
            'p': p.tolist(), 'real': real.tolist()}

def train(activation, seed, lr, ee, max_epochs):
    model = MLP(activation, seed)
    x, targets = normalize(RAW_X), normalize(RAW_Y)
    initial_dead = model.dead_units()
    first_dead = 0 if initial_dead else None
    history = [loss_sum(model, x, targets)]
    while history[-1] > ee and len(history) - 1 < max_epochs:
        for row, target in zip(x, targets):
            model.step(row, target, lr)
        error = loss_sum(model, x, targets)
        if not np.isfinite(error):
            raise FloatingPointError('Обучение разошлось; уменьшите --lr.')
        history.append(error)
        if activation == 'relu' and first_dead is None and model.dead_units():
            first_dead = len(history) - 1
    result = metrics(model)
    result.update(seed=seed, epochs=len(history)-1, converged=result['Es'] <= ee,
                  history=history, initial_dead=initial_dead,
                  final_dead=model.dead_units(), first_dead_epoch=first_dead,
                  weights=[a.tolist() for a in model.parameters()])
    return model, result

def predict(model, a, b):
    pair = np.asarray([a, b], dtype=float)
    if not np.all(np.isfinite(pair)) or np.any(np.abs(pair) > 10):
        raise ValueError('A и B должны быть конечными числами из [-10; 10].')
    p = float(model.forward(normalize(pair))[3])
    real = float(decode(p))
    return {'A': float(a), 'B': float(b), 'p': p,
            'real': real, 'class': float(nearest(real))}

def summarize(rows):
    successful = [r['epochs'] for r in rows if r['converged']]
    return {'successes': len(successful), 'runs': len(rows),
            'epochs_all_mean': float(np.mean([r['epochs'] for r in rows])),
            'epochs_success_mean': float(np.mean(successful)) if successful else None,
            'epochs_success_std': float(np.std(successful)) if successful else None,
            'epochs_success_min': min(successful) if successful else None,
            'epochs_success_max': max(successful) if successful else None,
            'accuracy_mean': float(np.mean([r['accuracy'] for r in rows])),
            'Es_mean': float(np.mean([r['Es'] for r in rows])),
            'mae_mean': float(np.mean([r['mae'] for r in rows])),
            'mae_std': float(np.std([r['mae'] for r in rows]))}

```

```

def visualize(results, models, ee, out, show):
    import matplotlib
    if not show:
        matplotlib.use('Agg')
    import matplotlib.pyplot as plt
    plt.rcParams.update({'font.family': 'DejaVu Sans', 'font.size': 10})
    colors = ['#2466a4', '#bf4c24']
    def save(fig, name):
        fig.savefig(out / name, dpi=180, bbox_inches='tight')
    fig, ax = plt.subplots(figsize=(8, 4), layout='constrained')
    for (act, rows), color in zip(results.items(), colors):
        ax.semilogy(rows[0]['history'], label=NAMES[act], color=color)
        ax.axhline(ee, color='black', ls='--', label=f'Ee = {ee:g}')
        ax.set(xlabel='Эпоха', ylabel='Суммарная BCE, Es',
               title=f'Сходимость, seed = {results["sigmoid"][0]["seed"]}')
        ax.legend(); ax.grid(alpha=.25); save(fig, 'convergence.png')
    fig, ax = plt.subplots(figsize=(8, 4), layout='constrained')
    pos = np.arange(len(results['sigmoid']))
    for k, (act, rows) in enumerate(results.items()):
        bars = ax.bar(pos + (k-.5)*.36, [r['epochs'] for r in rows],
                      .36, label=NAMES[act], color=colors[k])
        for bar, r in zip(bars, rows):
            if not r['converged']:
                bar.set_hatch('///')
            ax.annotate(str(r['epochs']),
                        (bar.get_x()+bar.get_width()/2, bar.get_height()),
                        ha='center', va='bottom', fontsize=8)
    ax.set_xticks(pos, [str(r['seed']) for r in results['sigmoid']])
    ax.set(xlabel='Seed', ylabel='Выполнено эпох',
           title='Штриховка: порог не достигнут')
    ax.margins(y=.35); ax.legend(); save(fig, 'epochs.png')
    grid = np.linspace(-10, 10, 301)
    aa, bb = np.meshgrid(grid, grid)
    fig, axes = plt.subplots(1, 2, figsize=(10, 4.6), layout='constrained')
    for ax, (act, model) in zip(axes, models.items()):
        values = decode(model.forward(normalize(np.c_[aa.ravel(), bb.ravel()])))
        values = values.reshape(aa.shape)
        im = ax.pcolormesh(aa, bb, values, cmap='viridis', vmin=-6, vmax=0,
                           shading='auto')
        if values.min() < -3 < values.max():
            ax.contour(aa, bb, values, levels=[-3], colors='white', linewidths=1.4)
        for cls, marker, color in [(0, 'o', 'red'), (-6, 's', 'cyan')]:
            pts = RAW_X[RAW_Y == cls]
            ax.scatter(*pts.T, marker=marker, c=color, s=65,
                       edgecolor='black', label=f'Цель {cls}')
        ax.plot([], [], color='white', label='Граница y = -3')
        ax.set(xlabel='A', ylabel='B', title=NAMES[act],
               xlim=(-10, 10), ylim=(-10, 10), aspect='equal')
        ax.legend(fontsize=8, facecolor='#dddddd', loc='upper right')
    fig.colorbar(im, ax=axes, label='Выход y = -6p', shrink=.8)
    save(fig, 'surfaces.png')
    fig, ax = plt.subplots(figsize=(7, 3.5), layout='constrained')
    bars = ax.bar(list(NAMES.values()),
                  [summarize(rows)['mae_mean'] for rows in results.values()], color=colors)
    ax.bar_label(bars, fmt='%.6f', padding=3)
    ax.set(ylabel='MAE в исходной шкале', title='Среднее по всем запускам')
    ax.margins(y=.2); save(fig, 'mae.png')
    if show:
        plt.show()
    plt.close('all')

def self_test():
    max_error = 0.
    for act in NAMES:
        for target in (0., 1.):
            model = MLP(act, 42)
            # Явно проверяются активная и неактивная ветви ReLU, вдали от 0.
            model.b1[:] = [.8, -1.2]
            x = np.array([.2, .8])
            grads = model.gradients(x, target)
            for param, grad in zip(model.parameters(), grads):

```

```

        for idx in np.ndindex(param.shape):
            old, eps = float(param[idx]), 1e-6
            param[idx] = old + eps
            plus = loss_sum(model, x, target)
            param[idx] = old - eps
            minus = loss_sum(model, x, target)
            param[idx] = old
            max_error = max(max_error, abs((plus-minus)/(2*eps)-grad[idx]))
_, a = train(act, 42, .5, .01, 10)
_, b = train(act, 42, .5, .01, 10)
assert a == b and a['epochs'] == 10
assert a['Es'] == a['history'][-1]
_, early = train(act, 42, .5, 100., 10)
assert early['epochs'] == 0 and early['converged']
assert max_error < 1e-7, max_error
assert np.array_equal(decode(normalize(RAW_Y)), RAW_Y)
assert float(nearest(-3.)) == C0
model = MLP('relu', 42)
model.w1[:] = 0.; model.b1[:] = 0.
assert model.dead_units() == [1, 2]
assert not np.any(model.gradients(np.ones(2), 1.)[1])
before = [p.copy() for p in model.parameters()[:3]]
model.step(np.ones(2), 1., .5)
assert all(np.array_equal(a, b) for a, b in zip(before, model.parameters()))
for bad in [(float('nan'), 0), (0, float('inf')), (-10.01, 0)]:
    try:
        predict(model, *bad)
    except ValueError:
        pass
    else:
        raise AssertionError('Недопустимый ввод принят')
predict(model, -10., 10.)
for z in (-1000., 1000.):
    model.w2[:] = 0.; model.b2[...] = z
    for target in (0., 1.):
        assert np.isfinite(loss_sum(model, np.zeros(2), target))
a, b = MLP('sigmoid', 42), MLP('relu', 42)
assert all(np.array_equal(x, y) for x, y in zip(a.parameters(), b.parameters()))
# Конструктивная проверка выразительности ReLU 2-2-1 для XOR.
model.w1[:] = [[1., 1.], [1., 1.]]
model.b1[:] = [0., -1.]
model.w2[:] = [20., -40.]; model.b2[...] = -10.
assert metrics(model)['accuracy'] == 1.
print(f'Проверки пройдены; ошибка градиента {max_error:.3e}')

```

```

def main():
    parser = argparse.ArgumentParser(description=__doc__,
                                     formatter_class=argparse.RawDescriptionHelpFormatter)
    parser.add_argument('--seeds', type=int, nargs='+', default=[42, 43, 44, 45, 46])
    parser.add_argument('--lr', type=float, default=.5)
    parser.add_argument('--ee', type=float, default=.01)
    parser.add_argument('--max-epochs', type=int, default=50000)
    parser.add_argument('--out', type=Path,
                        default=Path(__file__).resolve().parent / 'lab3_results')
    parser.add_argument('--no-input', action='store_true')
    parser.add_argument('--no-show', action='store_true')
    parser.add_argument('--self-test', action='store_true')
    args = parser.parse_args()
    if args.self_test:
        self_test(); return
    if (not np.isfinite(args.lr) or args.lr <= 0 or not np.isfinite(args.ee)
        or args.ee <= 0 or args.max_epochs < 1 or min(args.seeds) < 0
        or len(set(args.seeds)) != len(args.seeds) or len(args.seeds) < 5):
        parser.error('Нужны >=5 разных seed>=0, конечные lr>0, ee>0, max-epochs>=1.')
    args.out.mkdir(parents=True, exist_ok=True)
    results, models = {}, {}
    for act in NAMES:
        results[act] = []
        for seed in args.seeds:
            model, r = train(act, seed, args.lr, args.ee, args.max_epochs)
            results[act].append(r)
            if seed == args.seeds[0]:

```

```

        models[act] = model
    print(f'{NAMES[act]:14} seed={seed} эпох={r["epochs"]:5d} '
          f'Es={r["Es"]:.9f} acc={r["accuracy"]:.0%} MAE={r["mae"]:.6f} '
          f'noPOR={r["converged"]} мёртвые={r["final_dead"]}', flush=True)
    stats = {act: summarize(rows) for act, rows in results.items()}
    checks = {act: [predict(models[act], *pair) for pair in CHECK_X] for act in NAMES}
    data = {'parameters': {'seeds': args.seeds, 'lr': args.lr, 'ee': args.ee,
                           'max_epochs': args.max_epochs},
            'numpy_version': np.__version__, 'results': results,
            'summary': stats, 'checks': checks}
    (args.out / 'results.json').write_text(
        json.dumps(data, ensure_ascii=False, indent=2, allow_nan=False), encoding='utf-8')
    with (args.out / 'summary.csv').open('w', encoding='utf-8-sig', newline='') as f:
        writer = csv.DictWriter(f, fieldnames=['activation', *stats['sigmoid']])
        writer.writeheader()
        for act, row in stats.items():
            writer.writerow({'activation': act, **row})
    for act, row in stats.items():
        print(f'\n{NAMES[act]}: noPOR {row["successes"]}/{row["runs"]}; '
              f'средняя MAE={row["mae_mean"]:.6f}; '
              f'среднее эпох успешных={row["epochs_success_mean"]}')
    for act, rows in checks.items():
        print('\n' + NAMES[act] + f', seed={args.seeds[0]}')
        for r in rows:
            print(f'A={r["A"]:.6f}, B={r["B"]:.6f}: p={r["p"]:.6f}, '
                  f'y={r["real"]:.6f}, класс={r["class"]:.6f}')
    visualize(results, models, args.ee, args.out, not args.no_show)
    print(f'Результаты и графики: {args.out.resolve()}')
    if args.no_input:
        return
    print('Введите A B из [-10; 10], q – выход. Для новых пар эталон не задан.')
    while True:
        try:
            line = input('A B > ').strip()
        except (EOFError, KeyboardInterrupt):
            print(); break
        if line.lower() in ('q', 'quit', 'exit', 'выход'):
            break
        try:
            vals = list(map(float, line.replace(' ', '').split()))
            if len(vals) != 2:
                raise ValueError('Введите ровно два числа через пробел.')
            for act in NAMES:
                r = predict(models[act], *vals)
                print(f'{NAMES[act]}: p={r["p"]:.6f}, y={r["real"]:.6f}, '
                      f'класс={r["class"]:.6f}')
        except ValueError as exc:
            print(f'Ошибка ввода: {exc}')

if __name__ == '__main__':
    main()

```